Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5
По дисциплине: «Современные платформы программирования»

Выполнил:
Студент 3 курса
Группы ПО-12
Ишимов А.В.
Проверила:
Кулик А.Д.

Брест 2026

Вариант 7

Цель работы: приобрести практические навыки разработки API и баз данных.

Задание:

1. Реализовать базу данных из не менее 5 таблиц на заданную тематику. При реализации продумать типизацию полей и внешние ключи в таблицах;

2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними;

3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;

4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);

5. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпойнт;

База данных Компьютерная лаборатория

Код программы

```
from fastapi import FastAPI, HTTPException
from sqlalchemy import create_engine, Column, Integer, String, Boolean, DateTime, ForeignKey
from sqlalchemy.orm import sessionmaker, declarative_base
from datetime import datetime

# Подключение к MySQL
DATABASE_URL = "mysql+pymysql://root:root@localhost/computer_lab"

engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(bind=engine)
Base = declarative_base()

app = FastAPI(title="Computer Lab API")


class Laboratory(Base):
    __tablename__ = "Laboratories"

    id = Column(Integer, primary_key=True)
    name = Column(String(100), nullable=False)
    room_number = Column(String(20), nullable=False)
    capacity = Column(Integer, nullable=False)


class Workstation(Base):
    __tablename__ = "Workstations"
```

```
    id = Column(Integer, primary_key=True)
    inventory_number = Column(String(50), nullable=False, unique=True)
    lab_id = Column(Integer, ForeignKey("Laboratories.id"), nullable=False)
    cpu = Column(String(100), nullable=False)
    ram_gb = Column(Integer, nullable=False)
    has_gpu = Column(Boolean, nullable=False, default=False)
    os = Column(String(50), nullable=False)


class User(Base):
    __tablename__ = "Users"

    id = Column(Integer, primary_key=True)
    full_name = Column(String(100), nullable=False)
    role = Column(String(50), nullable=False)
    email = Column(String(100), nullable=False, unique=True)


class Software(Base):
    __tablename__ = "Software"

    id = Column(Integer, primary_key=True)
    name = Column(String(100), nullable=False)
    vendor = Column(String(100))
    version = Column(String(50))
    license_type = Column(String(50))


class Session(Base):
    __tablename__ = "Sessions"

    id = Column(Integer, primary_key=True)
    user_id = Column(Integer, ForeignKey("Users.id"), nullable=False)
    workstation_id = Column(Integer, ForeignKey("Workstations.id"), nulla-
ble=False)
    start_time = Column(DateTime, nullable=False, default=datetime.utcnow)
    end_time = Column(DateTime, nullable=True)
    purpose = Column(String(200))


Base.metadata.create_all(engine)


@app.post("/labs/")
def create_lab(name: str, room_number: str, capacity: int):
    db = SessionLocal()
    lab = Laboratory(name=name, room_number=room_number, capacity=capacity)
    db.add(lab)
    db.commit()
    return {"message": "Laboratory added"}


@app.get("/labs/")
def get_labs():
    db = SessionLocal()
    return db.query(Laboratory).all()


@app.put("/labs/{lab_id}")
def update_lab(lab_id: int, name: str = None, room_number: str = None,
capacity: int = None):
    db = SessionLocal()
    lab = db.query(Laboratory).get(lab_id)
```

```
        if not lab:
            raise HTTPException(status_code=404, detail="Laboratory not found")
        if name:
            lab.name = name
        if room_number:
            lab.room_number = room_number
        if capacity is not None:
            lab.capacity = capacity
        db.commit()
        return {"message": "Laboratory updated"}


@app.delete("/labs/{lab_id}")
def delete_lab(lab_id: int):
    db = SessionLocal()
    lab = db.query(Laboratory).get(lab_id)
    if not lab:
        raise HTTPException(status_code=404, detail="Laboratory not found")
    db.delete(lab)
    db.commit()
    return {"message": "Laboratory deleted"}


@app.post("/workstations/")
def create_workstation(
    inventory_number: str, lab_id: int, cpu: str, ram_gb: int, has_gpu: bool
= False, os: str = "Windows"
):
    db = SessionLocal()
    ws  =  Workstation(inventory_number=inventory_number,  lab_id=lab_id,
cpu=cpu, ram_gb=ram_gb, has_gpu=has_gpu, os=os)
    db.add(ws)
    db.commit()
    return {"message": "Workstation added"}


@app.get("/workstations/")
def get_workstations():
    db = SessionLocal()
    return db.query(Workstation).all()


@app.put("/workstations/{ws_id}")
def update_workstation(
    ws_id: int,
    inventory_number: str = None,
    lab_id: int = None,
    cpu: str = None,
    ram_gb: int = None,
    has_gpu: bool = None,
    os: str = None,
):
    db = SessionLocal()
    ws = db.query(Workstation).get(ws_id)
    if not ws:
        raise   HTTPException(status_code=404,   detail="Workstation   not
found")
    if inventory_number:
        ws.inventory_number = inventory_number
    if lab_id:
        ws.lab_id = lab_id
    if cpu:
        ws.cpu = cpu
    if ram_gb is not None:
```

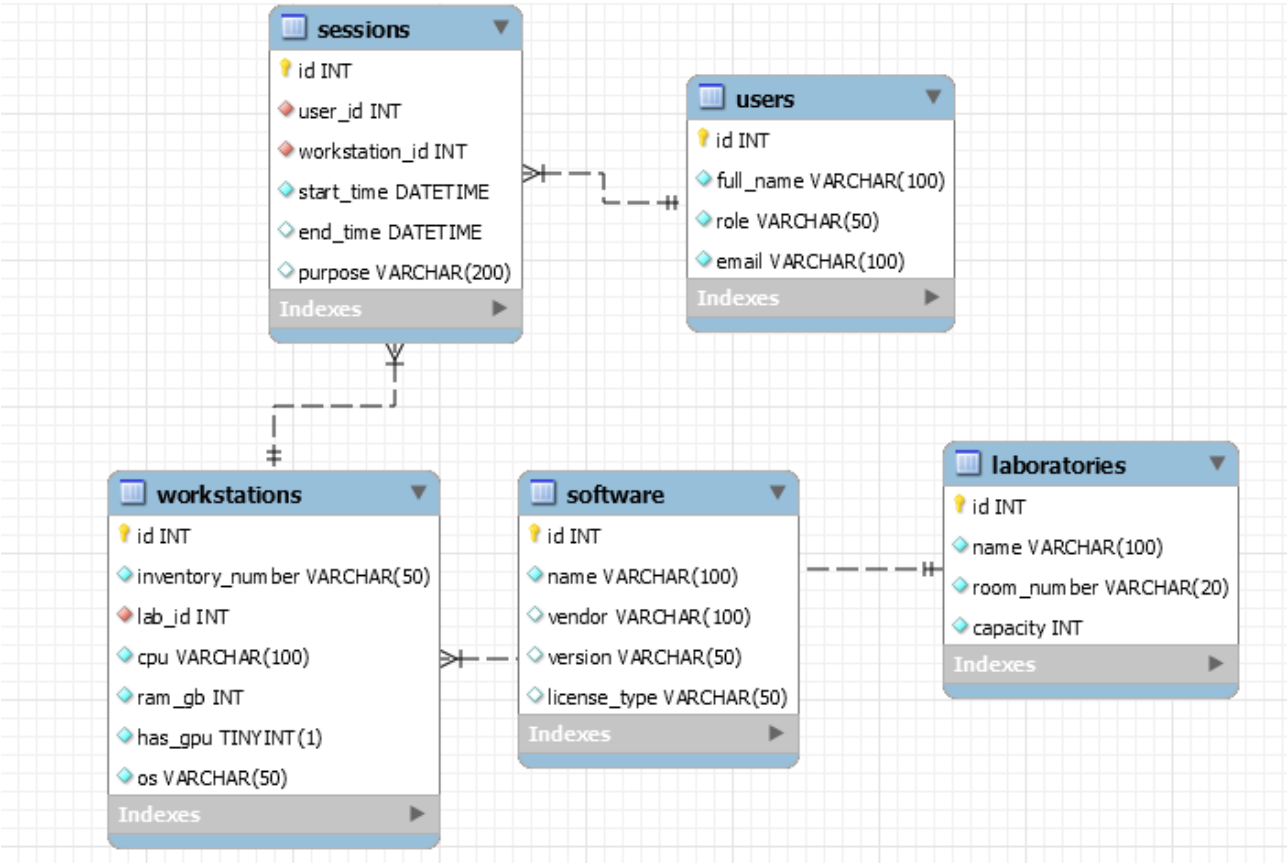
```
            ws.ram_gb = ram_gb
        if has_gpu is not None:
            ws.has_gpu = has_gpu
        if os:
            ws.os = os
        db.commit()
        return {"message": "Workstation updated"}


    @app.delete("/workstations/{ws_id}")
    def delete_workstation(ws_id: int):
        db = SessionLocal()
        ws = db.query(Workstation).get(ws_id)
        if not ws:
            raise   HTTPException(status_code=404,   detail="Workstation   not
found")
        db.delete(ws)
        db.commit()
        return {"message": "Workstation deleted"}


    @app.post("/users/")
    def create_user(full_name: str, role: str, email: str):
        db = SessionLocal()
        u = User(full_name=full_name, role=role, email=email)
        db.add(u)
        db.commit()
        return {"message": "User added"}


    @app.get("/users/")
    def get_users():
        db = SessionLocal()
        return db.query(User).all()


    @app.put("/users/{user_id}")
    def update_user(user_id: int, full_name: str = None, role: str = None,
email: str = None):
        db = SessionLocal()
        u = db.query(User).get(user_id)
        if not u:
            raise HTTPException(status_code=404, detail="User not found")
        if full_name:
            u.full_name = full_name
        if role:
            u.role = role
        if email:
            u.email = email
        db.commit()
        return {"message": "User updated"}


    @app.delete("/users/{user_id}")
    def delete_user(user_id: int):
        db = SessionLocal()
        u = db.query(User).get(user_id)
        if not u:
            raise HTTPException(status_code=404, detail="User not found")
        db.delete(u)
        db.commit()
        return {"message": "User deleted"}
```

```
@app.post("/software/")
def create_software(name: str, vendor: str = None, version: str = None, license_type: str = None):
    db = SessionLocal()
    s = Software(name=name, vendor=vendor, version=version, license_type=license_type)
    db.add(s)
    db.commit()
    return {"message": "Software added"}


@app.get("/software/")
def get_software():
    db = SessionLocal()
    return db.query(Software).all()


@app.put("/software/{soft_id}")
def update_software(soft_id: int, name: str = None, vendor: str = None, version: str = None, license_type: str = None):
    db = SessionLocal()
    s = db.query(Software).get(soft_id)
    if not s:
        raise HTTPException(status_code=404, detail="Software not found")
    if name:
        s.name = name
    if vendor:
        s.vendor = vendor
    if version:
        s.version = version
    if license_type:
        s.license_type = license_type
    db.commit()
    return {"message": "Software updated"}


@app.delete("/software/{soft_id}")
def delete_software(soft_id: int):
    db = SessionLocal()
    s = db.query(Software).get(soft_id)
    if not s:
        raise HTTPException(status_code=404, detail="Software not found")
    db.delete(s)
    db.commit()
    return {"message": "Software deleted"}


@app.post("/sessions/")
def create_session(
    user_id: int, workstation_id: int, purpose: str = None, start_time: str = None, end_time: str = None
):
    db = SessionLocal()

    start_dt = datetime.fromisoformat(start_time) if start_time else datetime.utcnow()
    end_dt = datetime.fromisoformat(end_time) if end_time else None

    sess = Session(
        user_id=user_id, workstation_id=workstation_id, start_time=start_dt, end_time=end_dt, purpose=purpose
    )
    db.add(sess)
    db.commit()
```

```
    return {"message": "Session added"}


@app.get("/sessions/")
def get_sessions():
    db = SessionLocal()
    return db.query(Session).all()


@app.put("/sessions/{session_id}")
def update_session(
    session_id: int,
    user_id: int = None,
    workstation_id: int = None,
    purpose: str = None,
    start_time: str = None,
    end_time: str = None,
):
    db = SessionLocal()
    sess = db.query(Session).get(session_id)
    if not sess:
        raise HTTPException(status_code=404, detail="Session not found")

    if user_id:
        sess.user_id = user_id
    if workstation_id:
        sess.workstation_id = workstation_id
    if purpose:
        sess.purpose = purpose
    if start_time:
        sess.start_time = datetime.fromisoformat(start_time)
    if end_time:
        sess.end_time = datetime.fromisoformat(end_time)

    db.commit()
    return {"message": "Session updated"}


@app.delete("/sessions/{session_id}")
def delete_session(session_id: int):
    db = SessionLocal()
    sess = db.query(Session).get(session_id)
    if not sess:
        raise HTTPException(status_code=404, detail="Session not found")
    db.delete(sess)
    db.commit()
    return {"message": "Session deleted"}


# python -m uvicorn lab5:app --reload
```

Рисунки с результатами работы программы

```
PS C:\Users\Mr_Herceg\Desktop\spp_po12\reports\Ishimov_A_V\Lab5\src> python -m uvicorn lab5:app --reload
INFO:     Will watch for changes in these directories: ['C:\\Users\\Mr_Herceg\\Desktop\\spp_po12\\reports\\Ishimov_A_V\\Lab5\\src']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [14488] using StatReload
INFO:     Started server process [9080]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```



Вывод: приобрел практические навыки разработки API и баз данных.